DDS HR Chatbot — Render Deployment Guide

Sep 26, 2026 · @Mohammad Arshad

Deploying the DDS Enterprise HR Chatbot (Gradio + LlamaIndex + Pinecone) from the Hugging Face Space `hrenterprise` to Render, via GitHub repo `arshad831/ddshrent`.

Overview

- **Source:** Hugging Face Space `decodingdatascience/hrenterprise` — a Gradio app answering HR questions from synthetic DDS policy PDFs, using LlamaIndex + Pinecone + OpenAI (`gpt-4o-mini`, `text-embedding-ada-002`).
- **Why move it:** Hugging Face's Docker image auto-installs `gradio`, `openai`, and the Pinecone SDK based on the Space's `sdk: gradio` config. Render has no equivalent, so the app needed two fixes before it would run there.
- **New home:** GitHub repo `arshad831/ddshrent`, deployed on Render as a Web Service.
- **Status as of this doc:** repo pushed and connected to Render; Configure step in progress (screenshots below).

Decoding Data Science (DDS)

Deployment Steps

Part 1 — Code to GitHub

1. Unzip the Render-ready package (`app.py`, `requirements.txt`, `Data/` PDFs).
2. Create a GitHub repo (done: `arshad831/ddshrent`).
3. From the unzipped folder: `git init`, `git add .`, `git commit -m "Initial commit"`, `git branch -M main`, `git remote add origin <repo-url>`, `git push -u origin main`.

Part 2 — Render Web Service

1. Log into render.com (sign up with GitHub for a one-step connection).
2. Click **+ New → Web Service**.
3. Connect GitHub, authorize the Render GitHub App, select `ddshrent`.
4. On the Configure screen, set:
   - **Name:** `ddshrent` (or any name)

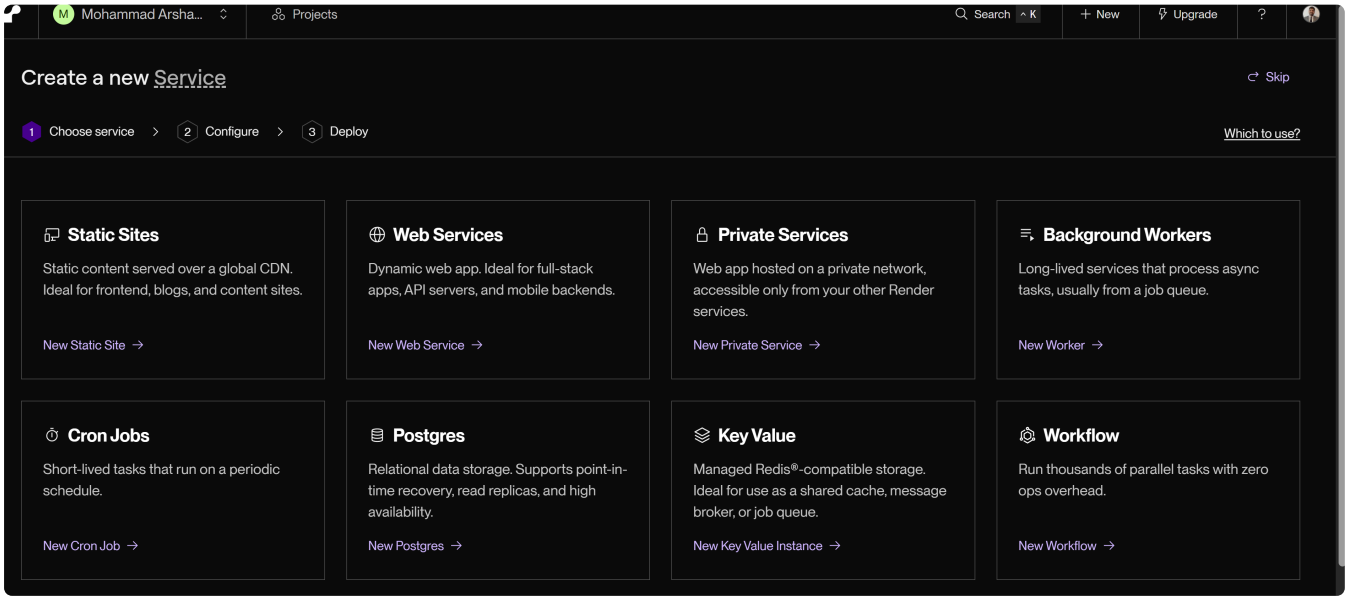
- **Language/Runtime:** Python 3
- **Branch:** `main`
- **Root Directory:** leave blank (`app.py` is at repo root)
- **Build Command:** `pip install -r requirements.txt`
- **Start Command:** `python app.py`
- **Instance Type:** Starter ($7/mo, 0.5 CPU, 512 MB RAM) — free tier tends to run out of memory loading LlamaIndex

5. Add the two environment variables (see below).
6. Click **Deploy web service**.
7. First boot builds the Pinecone index from the PDFs — allow 1–2 extra minutes past "Live" before testing.
8. Open the assigned URL (`https://<name>.onrender.com`) to confirm the Gradio chat UI loads.

Decoding Data Science (DDS)

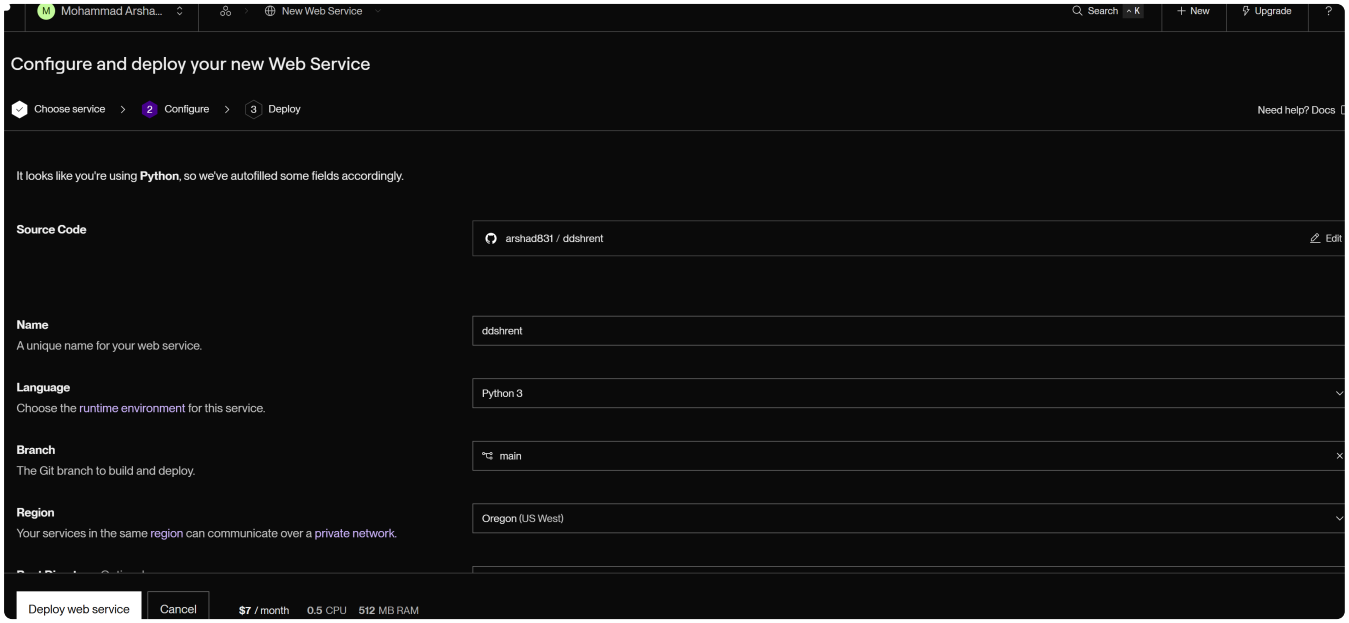
Screenshots

Choose Service



Render's "Create a new Service" screen. **Web Services** is the correct card for this app (dynamic Gradio app, not a Static Site/Private Service/Worker).

Configure and Deploy



Render auto-detected Python from the repo. Field values used on this screen:

Field	Value
Source Code	`arshad831/ddshrent`
Name	`ddshrent`
Language	Python 3
Branch	`main`
Region	Oregon (US West), or a region closer to Dubai if available
Root Directory	blank
Build Command	`pip install -r requirements.txt`
Start Command	`python app.py`
Instance Type	Starter — $7/month, 0.5 CPU, 512 MB RAM

Decoding Data Science (DDS)

Files & Changes vs. the Hugging Face Version

File	Change
`app.py`	Reads the port from Render's `PORT` env var instead of the hardcoded `7860`; added `.queue()` for concurrent requests
`requirements.txt`	Now lists `gradio`, `openai`, and `pinecone` explicitly — Hugging Face installed these automatically from the Space's `sdk` config, Render does not
`Data/*.pdf`	Same four HR policy PDFs (Employee Handbook, HR FAQ, Leave Policy v2, Remote Work Policy), unchanged content
`DEPLOY_NOTES.md`	New — Render-specific setup notes kept in the repo for reference

Repo: github.com/arshad831/ddshrent

Decoding Data Science (DDS)

Environment Variables (Render Dashboard)

Key	Value
`OPENAI_API_KEY`	Your OpenAI key (enter directly in Render, not shared here)
`PINECONE_API_KEY`	Your Pinecone key (enter directly in Render, not shared here)

Set under the service's **Environment** tab before or right after the first deploy. `app.py` reads both via `os.getenv(...)` and raises an error at startup if either is missing.

Decoding Data Science (DDS)

Open Item

- [ ] Confirm deploy succeeds and the Gradio chat UI loads at the assigned `.onrender.com` URL